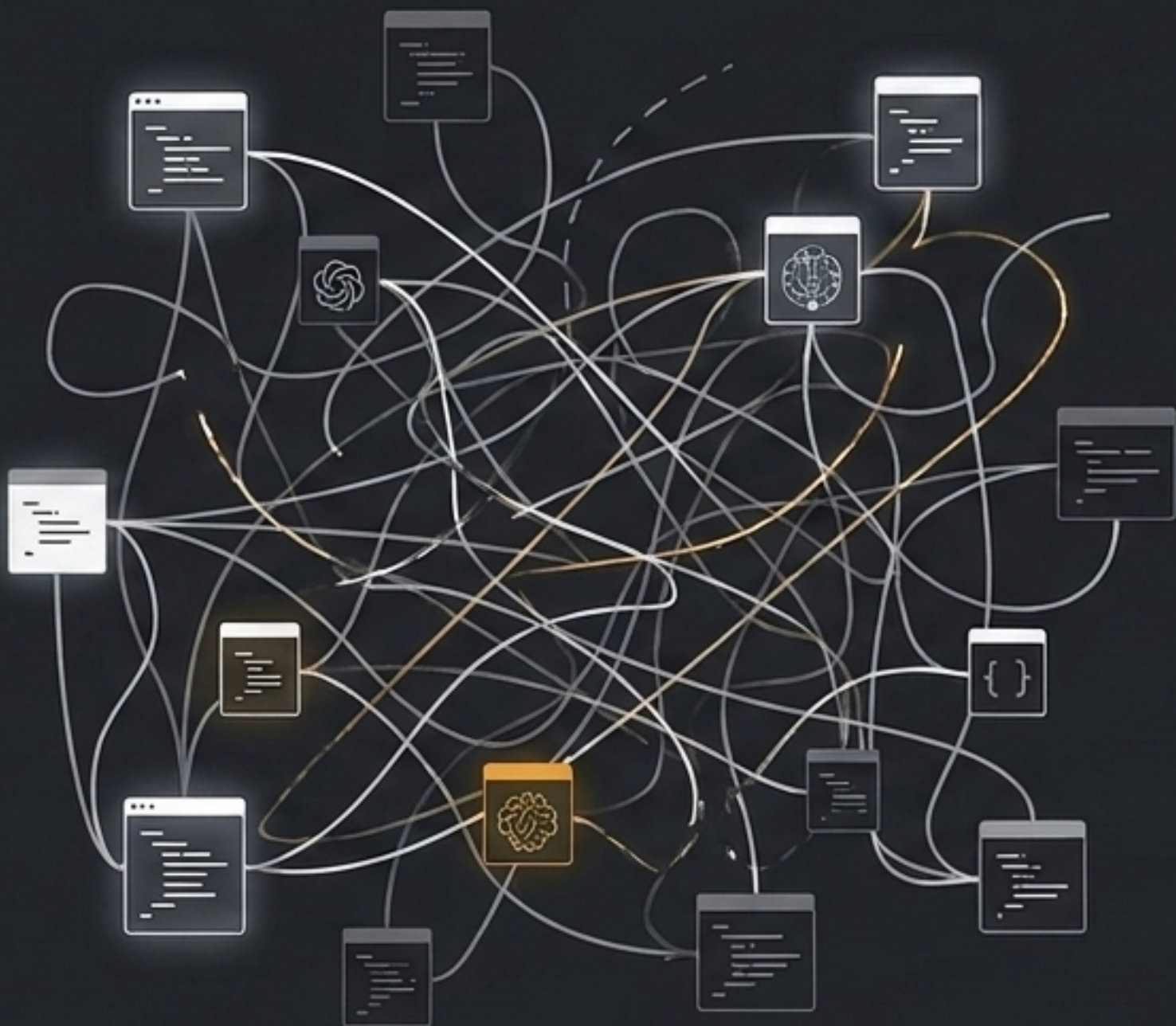


sdd-harness

Desarrollo guiado por especificaciones
para equipos con agentes de IA.

Código rápido. Cero gobernanza.



[WARN] Sin contexto global del proyecto.

[WARN] Sin trazabilidad de decisiones técnicas.

[WARN] Sin gates de calidad unificados.

[WARN] Cada desarrollador usa su propio flujo (prompts aislados).

El costo real aparece en el review, las regresiones y el conocimiento que se evapora al cerrar la sesión.

El paradigma SDD: Nada se implementa sin especificación aprobada.



Roles Estrictos

7 agentes especializados (Orquestador, Funcional, Planner, Arquitecto, Implementadores, Reviewer, Steward).

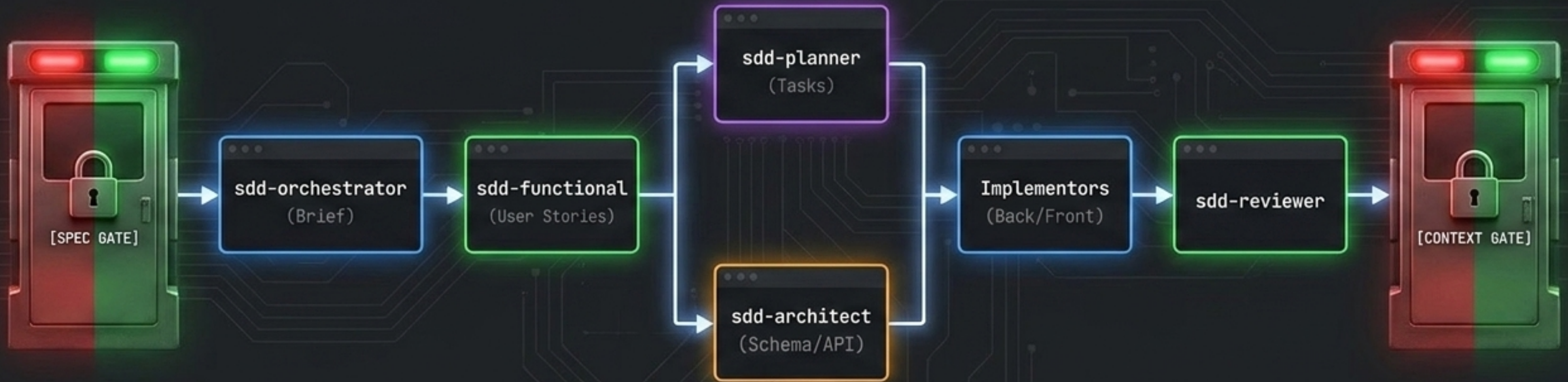
Artefactos Obligatorios

Registros JSON y Markdown validados antes de avanzar.

Gates Automáticos

Validaciones inquebrantables que bloquean el código no diseñado.

El Ciclo SDD en acción.



La única fuente de verdad: el directorio `sdd/`

`sdd/`

- `agents/ & skills/`
- `specs/`
- `context/`
- `*.json`
- `scripts/`
- `docs/`

El cerebro.

(global, api, tasks): Registros estrictos. La fuente de verdad.

Validador y generadores.

Visor interactivo local.

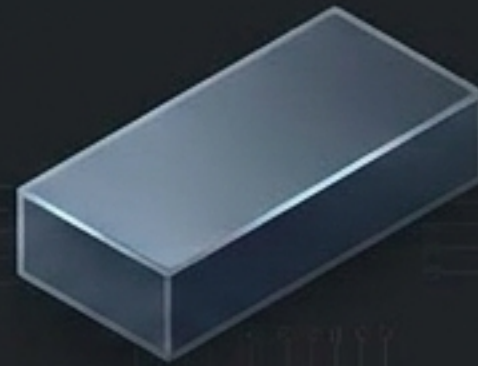
Todo versionado en TU repo. Actualizable (**`harness update sdd`**) sin pisar tu código.

Un sistema. Tres modos de entrada.



Monorepo Nx (Desde Cero)

- Múltiples apps y librerías compartidas.
- Configuración raíz derivada del kit portátil.



Standalone App (Desde Cero)

- Una única aplicación raíz (ej. Next.js, Fastify).
- Metodología idéntica sin la complejidad de Nx.



Proyecto Existente (Legacy)

- Se inyecta la carpeta **sdd/**.
- Cero cambios en el código **src/** existente.

Agenda de Demostración



Escenario 1 — Nx desde cero (flexi-kudos).

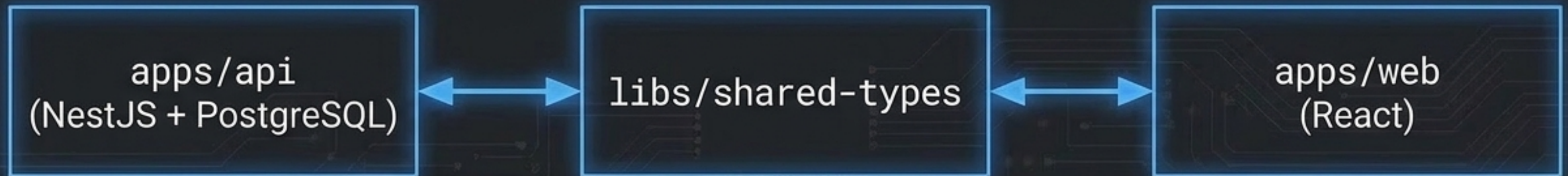


Escenario 2 — App Standalone (quick-polls).



Escenario 3 — Proyecto Existente (legacy-shop).

Escenario 1: Monorepo Nx desde cero



Proyecto:	flexi-kudos (Muro de reconocimientos del equipo).
Feature a construir:	Leaderboard semanal (Endpoint + Tipo + Widget).

→ PASAR A DEMO EN VIVO (30 min)

Qué mirar durante el ciclo

Perfil Funcional	Desarrolladores	CTOs / Líderes
Artefacto: functional.md	Artefactos: tasks.json & api.json	Artefacto: cycle.json
Foco: Historias de usuario y criterios de aceptación verificables.	Foco: Tareas atómicas y contratos de API validados antes de codear.	Foco: Estado auditable del ciclo y telemetría de ejecución.

Escenario 2: Standalone & El SPEC GATE

Proyecto:	quick-polls (Encuestas rápidas en Next.js).
Momento Clave:	El agente recibe una orden de implementación sin especificación previa.
Feature a construir:	Cerrar encuesta y ver gráfico de resultados.



NEXT.js

[SPEC GATE] : REJECTED

→ PASAR A DEMO EN VIVO (20 min)

Escenario 3: Proyecto Existente (Legacy)

Antes

```
.  
├─ src/ (código legacy)  
└─ package.json
```

Después de harness configure

```
.  
├─ src/ (código legacy intacto - 0 cambios)  
└─ package.json  
└─ sdd/ (+ sistema de gobernanza inyectado)
```

Proyecto: legacy-shop (Catálogo con carrito en Fastify + React).

Harness Dual/Multi: Claude Code + Copilot + Gemini leyendo de la misma fuente.

Feature a construir: Cupones de descuento (respetando patrones legacy).

→ PASAR A DEMO EN VIVO (20 min)

Gobernanza y Calidad Integrada



[LOCKED]

Gates obligatorios (SPEC y CONTEXT).



[STRICT]

JSON Schemas inquebrantables
(`additionalProperties: false`).



[CI/CD]

`pnpm sdd:validate` rompe el build si
no hay cumplimiento.



[FOCUS]

Un solo módulo in-progress a la vez
por desarrollador.

Telemetría y Visibilidad de Costos

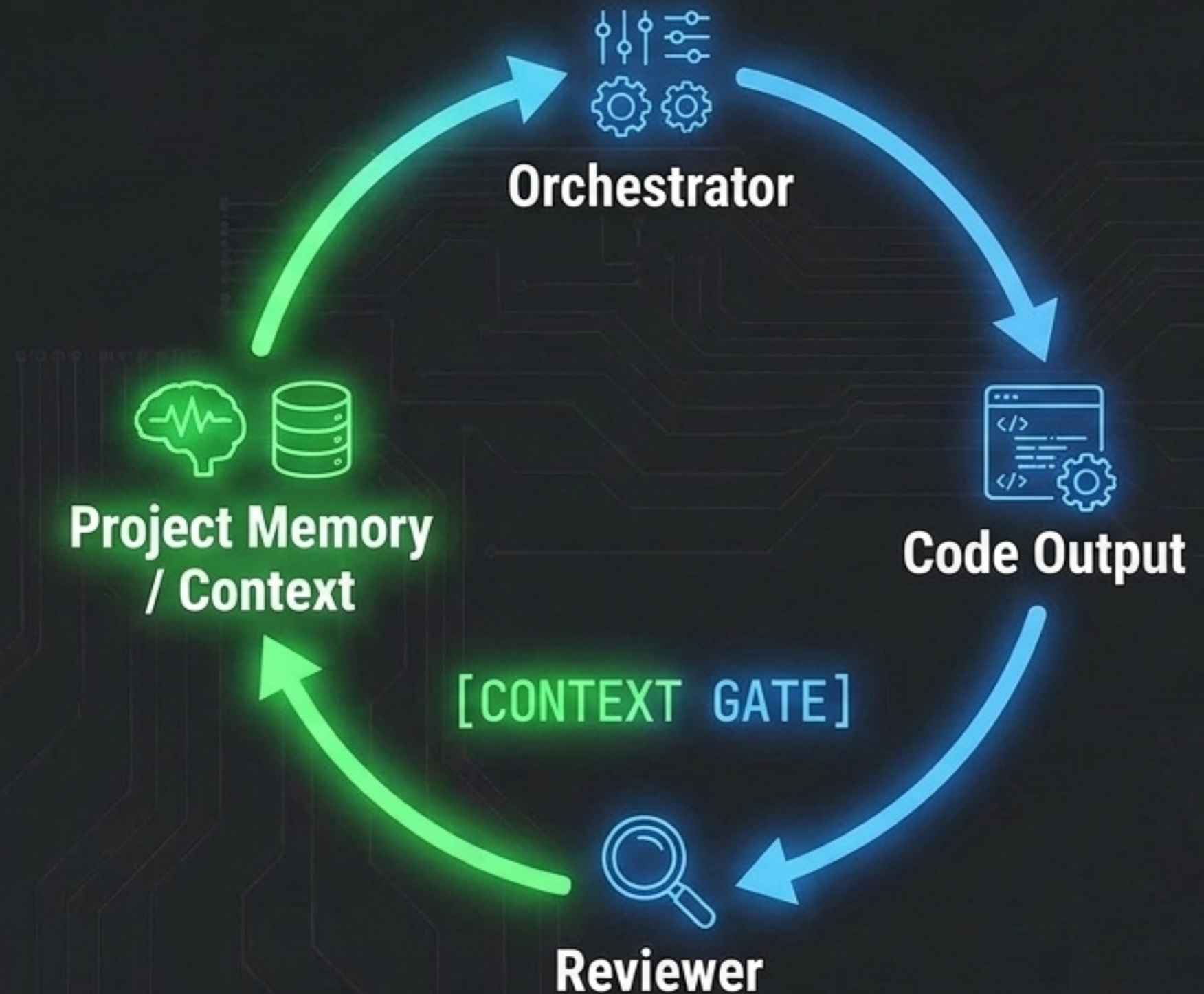
- ✓ **Monitoreo granular:** Seguimiento del consumo de tokens por cada agente (Claude, Copilot, Gemini).
- ✓ **Transparencia Financiera:** Visibilidad en tiempo real de los costos asociados al desarrollo asistido por IA.
- ✓ **Optimización de Recursos:** Identificación de áreas de alto consumo para ajustar estrategias.



*Los montos son ilustrativos y no representan costos reales. Ver `sdd/pricing.json` para configuración.

Memoria del Proyecto: Nunca pagar dos veces por el mismo error

- ✓ **Journal Episódico:** Los aprendizajes de cada ciclo quedan documentados en la bitácora.
- ✓ **EL CONTEXT GATE:** Obliga al Reviewer a actualizar el estado del subproyecto (sin merge conflicts gracias a fragmentos aditivos).
- ✓ El aprendizaje vuelve al contexto de los agentes para el próximo ciclo.



Adopción sin fricción

01

Evaluar (Cero Riesgo)

Revisar el repositorio público de ejemplos: e-burgos/sdd-harness-examples.
(Regenerado en cada release, sin instalar nada).

02

Piloto Técnico

Requisitos: Node $\geq$ 18.
Ejecutar harness
configure sdd en una rama de un proyecto existente.

03

Escalar

Proyectos nuevos →
Estándar metodológico del equipo.

sdd-harness

Desarrollo Rápido

Stack completo generado y listo para IA (Escenario 1 & 2).

Gobierno Estricto

Gates, validaciones CI/CD y trazabilidad total de diseño a código.

Adopción Transparente

Se integra sobre repositorios legacy sin alterar el código propio (Escenario 3).

Q&A